

Junções (JOINS) em SQL Server

Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária — SEFAZ-PB

Banco de trabalho: `curso_integridade_fiscal` **SGBD:** Microsoft SQL Server 2016 ou superior (T-SQL)

Ambiente: SQL Server Management Studio (SSMS) ou VS Code + extensão MSSQL

Sumário

1. Objetivos de aprendizagem
 2. Por que junções existem
 3. O mapa de relacionamentos da base
 4. Sintaxe da junção em T-SQL
 5. Produto cartesiano e CROSS JOIN
 6. INNER JOIN — a junção interna
 7. LEFT OUTER JOIN — preservando o lado esquerdo
 8. RIGHT OUTER JOIN — preservando o lado direito
 9. FULL OUTER JOIN — a conciliação bidirecional
 10. ON ou WHERE? O erro que anula a junção externa
 11. Antijunção e semijunção
 12. SELF JOIN — a tabela consigo mesma
 13. Junção theta (não equijunção)
 14. CROSS APPLY e OUTER APPLY
 15. Junção e agregação: o problema do fan-out
 16. NULL nas colunas de junção
 17. Junções e desempenho
 18. Erros comuns — quadro de diagnóstico
 19. Quadro resumo dos tipos de junção
 20. Roteiro prático de laboratório
 21. Referências
-

1. Objetivos de aprendizagem

Ao final deste módulo, o participante deverá ser capaz de:

- Explicar a junção como operação da álgebra relacional e relacioná-la ao produto cartesiano seguido de seleção;
- Escrever junções internas e externas em sintaxe ANSI, com aliases e critérios compostos;
- Escolher conscientemente entre `INNER`, `LEFT`, `RIGHT` e `FULL` conforme o lado da relação que precisa ser preservado na investigação fiscal;
- Distinguir o efeito de um predicado escrito na cláusula `ON` do mesmo predicado escrito na cláusula `WHERE`;
- Construir antijunções para localizar omissões (NF-e sem escrituração, contribuinte sem ordem de serviço);
- Aplicar autojunções para percorrer hierarquias e detectar operações circulares entre contribuintes;
- Usar junções theta contra a pauta de referência de preços;

- Reconhecer e corrigir a duplicação de linhas (*fan-out*) que inflaciona somatórios em cruzamentos de múltiplos níveis.

Pré-requisitos: módulos de Modelagem Relacional, Mapeamento MER→Relacional, Modelo Relacional (álgebra) e Restrições de Integridade. A junção é a materialização, em SQL, do operador de junção estudado na álgebra relacional.

2. Por que junções existem

O modelo relacional não guarda os dados de uma nota fiscal em um único lugar. A normalização distribui a informação: o emitente fica em **contribuinte**, o município do emitente fica em **municipio**, o cabeçalho do documento fica em **nfe**, os produtos ficam em **nfe_item**, e a declaração do próprio contribuinte sobre aquele documento fica em **efd_c100** e **efd_c170**.

Essa distribuição é uma virtude — evita redundância, evita anomalias de atualização e permite que cada fato seja registrado uma única vez. Mas cobra um preço na hora da consulta: **a informação precisa ser recomposta**. A junção é a operação que faz essa recomposição.

Na álgebra relacional, a junção é definida como um produto cartesiano seguido de uma seleção:

```
nfe ⋈ (nfe.id_emitente = contribuinte.id_contribuinte) contribuinte
= σ(nfe.id_emitente = contribuinte.id_contribuinte) (nfe × contribuinte)
```

Essa equivalência é conceitual, não operacional. O otimizador do SQL Server nunca constrói os $10.636 \times 60 = 638.160$ pares para depois descartar quase todos: ele usa índices e algoritmos físicos de junção. Mas a definição importa, porque explica o comportamento que mais assusta o iniciante — **quando o critério de junção está errado ou ausente, o que sobra é o produto cartesiano**.

Nota fiscal do conceito. Na auditoria, quase nenhuma pergunta relevante se responde com uma tabela só. "Quais contribuintes de Campina Grande emitiram notas acima da pauta e não escrituraram na EFD?" envolve **municipio**, **contribuinte**, **nfe**, **nfe_item**, **referencia_preco** e **efd_c100**. Dominar junções é, na prática, dominar o cruzamento fiscal.

3. O mapa de relacionamentos da base

Antes de escrever qualquer junção, é preciso saber por onde as tabelas se ligam. O quadro abaixo é o mapa de chaves estrangeiras de **curso_integridade_fiscal**:

Tabela filha	Coluna(s)	Tabela pai	Coluna(s)	Cardinalidade	Obrigatoriedade
contribuinte	id_municipio	municipio	id_municipio	N:1	Sim, não nulo
auditor_fiscal	matricula_supervisor	auditor_fiscal	matricula	N:1 (autorrelacionamento)	Não, pode ser nulo
nfe	id_emitente	contribuinte	id_contribuinte	N:1	Sim, não nulo
nfe	id_destinatario	contribuinte	id_contribuinte	N:1	Não, pode ser nulo

Tabela filha	Coluna(s)	Tabela pai	Coluna(s)	Cardinalidade	Obrigatoriedade
nfe_item	id_nfe	nfe	id_nfe	N:1	Sim, deletar o filho também deleta o pai
efd_c100	id_contribuinte	contribuinte	id_contribuinte	N:1	Sim, deletar o filho também deleta o pai
efd_c170	id_c100	efd_c100	id_c100	N:1	Sim, deletar o filho também deleta o pai
ordem_servico	id_contribuinte	contribuinte	id_contribuinte	N:1	Sim, deletar o filho também deleta o pai
ordem_servico	matricula_auditor	auditor_fiscal	matricula	N:1	Sim, deletar o filho também deleta o pai
auto_infracao	id_os	ordem_servico	id_os	N:1	Sim, deletar o filho também deleta o pai
auto_infracao	id_contribuinte	contribuinte	id_contribuinte	N:1	Sim, deletar o filho também deleta o pai
socio_administrador	id_contribuinte	contribuinte	id_contribuinte	N:1	Sim, deletar o filho também deleta o pai

E há uma ligação que não é chave estrangeira e que é o coração deste módulo:

Coluna	Coluna	Situação
nfe.chave_acesso	efd_c100.chave_acesso	Não existe FK entre elas — propositalmente

Como o script `01_criar_banco.sql` documenta, a EFD pode escriturar documentos emitidos em outras UFs, ausentes da base local. Se houvesse chave estrangeira, o banco recusaria a escrituração legítima de um documento de fora e, pior, tornaria **impossível** a existência das inconsistências que a malha fiscal procura. É exatamente nessa fronteira sem restrição que as junções externas fazem seu trabalho.

Volumes da base carregada (conferidos pelo script `07_verificacao.sql`):

Tabela	Linhas
municipio	35
contribuinte	60
auditor_fiscal	20
referencia_preco	28
nfe	10.636
nfe_item	30.794
efd_c100	8.059
efd_c170	23.158
ordem_servico	150
auto_infracao	72
socio_administrador	138 (script 09)

4. Sintaxe da junção em T-SQL

4.1 A forma ANSI (a única que se deve usar)

```
SELECT <colunas>
FROM   tabela_a AS a
```

```
<tipo> JOIN tabela_b AS b ON <predicado de junção>
WHERE <filtros de linha>;
```

Os tipos disponíveis:

```
[INNER] JOIN          -- interna (INNER é opcional; escreva-o mesmo assim)
LEFT [OUTER] JOIN     -- externa à esquerda
RIGHT [OUTER] JOIN    -- externa à direita
FULL [OUTER] JOIN     -- externa completa
CROSS JOIN            -- produto cartesiano
CROSS APPLY / OUTER APPLY -- junção com subconsulta correlacionada
```

A palavra **OUTER** é opcional e não muda nada: **LEFT JOIN** e **LEFT OUTER JOIN** são idênticos. A palavra **INNER** também é opcional, mas **escrevê-la é boa prática**: deixa explícito para quem lê o script — inclusive para o auditor que vai revisar o trabalho meses depois — que a omissão de linhas sem par foi uma decisão, não um esquecimento.

4.2 A forma antiga (a que se deve reconhecer, mas não escrever)

```
-- Junção interna na sintaxe implícita (ainda funciona, mas evite)
SELECT n.numero, c.razao_social
FROM   dbo.nfe AS n, dbo.contribuinte AS c
WHERE  n.id_emitente = c.id_contribuinte;
```

Funciona, produz o mesmo resultado, e ainda aparece em scripts legados da própria repartição. O problema é que **o critério de junção e o filtro de negócio se misturam na mesma cláusula WHERE**. Numa consulta com seis tabelas e doze predicados, esquecer um critério de junção passa despercebido — e o resultado é um produto cartesiano parcial que ninguém nota até os valores somados ficarem absurdos.

Já a sintaxe antiga de junção **externa**, com ***=** e **=***, foi **descontinuada** e não funciona mais em níveis de compatibilidade modernos:

```
-- NÃO FUNCIONA em SQL Server moderno. Está aqui apenas para reconhecimento.
SELECT ... FROM nfe n, efd_c100 e WHERE n.chave_acesso *= e.chave_acesso;
```

Se você encontrar isso num script herdado, a tarefa é reescrever em **LEFT JOIN**.

4.3 Aliases: uma regra de higiene

Sempre dê alias às tabelas e **sempre qualifique as colunas**:

```
-- Ruim: de qual tabela vem 'ncm'? E 'valor_icms'?
SELECT ncm, valor_icms FROM nfe_item JOIN efd_c170 ON ...

-- Bom
SELECT i.ncm, i.valor_icms_item, c170.ncm, c170.valor_icms_item
FROM   dbo.nfe_item AS i
       INNER JOIN dbo.efd_c170 AS c170 ON ...
```

Neste curso o padrão é: **n** = **nfe**, **i** = **nfe_item**, **e** = **efd_c100**, **c170** = **efd_c170**, **c** = **contribuinte**, **m** = **municipio**, **a** = **auditor_fiscal** ou **auto_infracao** (conforme o contexto), **o** = **ordem_servico**, **r** = **referencia_preco**.

5. Produto cartesiano e CROSS JOIN

O **CROSS JOIN** combina cada linha da esquerda com cada linha da direita. Não tem cláusula **ON**.

```
-- 35 municípios x 4 regimes = 140 linhas
SELECT m.nome, r.regime
FROM   dbo.municipio AS m
       CROSS JOIN (VALUES ('NORMAL'),('SIMPLES'),('MEI'),('ISENTO')) AS r(regime);
```

Parece inútil, e na maior parte do tempo é — quando aparece sem querer, é um bug. Mas tem um uso legítimo e recorrente na fiscalização: **gerar a grade completa de combinações que *deveriam* existir**, para depois confrontá-la com o que existe de fato.

```
-- Grade de referência: todo contribuinte ATIVO do regime NORMAL
-- x todos os 12 períodos de apuração de 2025.
-- Esta é a "lista de chamada" da obrigação acessória.
SELECT c.id_contribuinte, c.razao_social, p.periodo_apuracao
FROM   dbo.contribuinte AS c
       CROSS JOIN (VALUES ('202501'),('202502'),('202503'),('202504'),
                          ('202505'),('202506'),('202507'),('202508'),
                          ('202509'),('202510'),('202511'),('202512'))
                          AS p(periodo_apuracao)
WHERE  c.situacao_cadastral = 'ATIVO'
AND    c.regime_tributario  = 'NORMAL';
```

Guardada essa grade, basta uma junção externa contra **efd_c100** para descobrir **qual contribuinte deixou de entregar qual período** — o clássico levantamento de omissos. Voltaremos a isso na seção 11 e no laboratório.

⚠ **Cuidado com a escala.** **nfe CROSS JOIN nfe_item** produziria $10.636 \times 30.794 \approx 327$ milhões de linhas. Nunca execute um produto cartesiano sobre as tabelas de documentos desta base.

6. INNER JOIN — a junção interna

6.1 Semântica

O **INNER JOIN** retorna **somente as linhas em que o predicado **ON** é verdadeiro em ambos os lados**. Linha sem par de um lado ou do outro simplesmente desaparece do resultado.

```
Esquerda: A B C D      INNER JOIN em X:
Direita:   B C E        resultado = B C
```

Essa é a junção mais usada — e é justamente por isso que ela é perigosa numa auditoria. **Toda linha que some numa junção interna é uma informação perdida em silêncio**. Se você quer saber quem *não* declarou, o **INNER JOIN** nunca vai lhe dizer, porque quem não declarou não tem par do outro lado.

6.2 Exemplo mínimo: nota fiscal com o nome do emitente

```
USE curso_integridade_fiscal;
GO

SELECT TOP (10)
    n.numero,
    n.data_emissao,
    n.valor_total,
    c.cnpj,
```

```

c.razao_social
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c
              ON c.id_contribuinte = n.id_emitente
WHERE  n.situacao = 'AUTORIZADA'
ORDER BY n.data_emissao DESC;

```

Leitura da consulta: para cada linha de **nfe**, procure em **contribuinte** a linha cujo **id_contribuinte** seja igual ao **id_emitente**; junte as duas lado a lado.

Como **nfe.id_emitente** é **NOT NULL** e tem chave estrangeira para **contribuinte**, o banco garante que **todas** as 10.636 notas encontram emitente. Nesse caso específico, **INNER JOIN** e **LEFT JOIN** produzem exatamente o mesmo resultado — a integridade referencial já eliminou a possibilidade de órfãos. Essa é a primeira regra prática:

Onde existe FK obrigatória, **INNER** e **LEFT** coincidem. Onde a coluna é anulável ou não há FK, a escolha entre os dois muda o resultado.

6.3 Cadeia de junções: subindo até o município

```

SELECT TOP (20)
    c.cnpj,
    c.razao_social,
    m.nome AS municipio,
    m.uf,
    m.regiao_fiscal,
    SUM(n.valor_total) AS total_emitido
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       INNER JOIN dbo.municipio AS m ON m.id_municipio = c.id_municipio
WHERE  n.situacao = 'AUTORIZADA'
       AND n.tipo_operacao = '1' -- saída
       AND m.uf = 'PB'
GROUP BY c.cnpj, c.razao_social, m.nome, m.uf, m.regiao_fiscal
ORDER BY total_emitido DESC;

```

As junções são avaliadas conceitualmente da esquerda para a direita: primeiro **nfe** com **contribuinte**, depois o resultado disso com **municipio**. Nada impede o otimizador de escolher outra ordem física — o resultado é o mesmo.

6.4 Descendo ao item e chegando à EFD: o cruzamento de quatro níveis

Este é o cruzamento central da malha fiscal desta base: comparar, item a item, o que a NF-e diz com o que o contribuinte escreveu.

```

SELECT  n.chave_acesso,
        i.num_item,
        i.descricao,
        i.ncm AS ncm_nfe,
        c170.ncm AS ncm_efd,
        i.cfop AS cfop_nfe,
        c170.cfop AS cfop_efd,
        i.valor_total_item AS valor_nfe,
        c170.valor_item AS valor_efd,
        i.valor_total_item - c170.valor_item AS diferenca
FROM     dbo.nfe AS n
        INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
        INNER JOIN dbo.efd_c100 AS e ON e.chave_acesso = n.chave_acesso
        INNER JOIN dbo.efd_c170 AS c170 ON c170.id_c100 = e.id_c100
        AND c170.num_item = i.num_item
WHERE    n.situacao = 'AUTORIZADA'
        AND e.cod_situacao = '00'
        AND e.ind_oper = '1'
        AND (i.ncm <> c170.ncm

```

```
OR i.cfop < c170.cfop
OR ABS(i.valor_total_item - c170.valor_item) > 0.01);
```

Dois detalhes merecem atenção:

1. A junção **efd_c170** tem **predicado composto** (**id_c100** e **num_item**). Sem o **num_item**, cada item da nota se cruzaria com *todos* os itens do documento escriturado — um produto cartesiano local que multiplicaria as linhas e destruiria qualquer somatório.
2. A junção com **efd_c100** é por **chave_acesso**, não por FK. É uma junção "de negócio", não uma junção estrutural. Só faz sentido porque a chave de acesso é o identificador nacional único do documento — e é por isso que o próprio script 07 verifica se existem chaves internamente inconsistentes (13 casos plantados na base).

O script **07_verificacao.sql** informa o resultado esperado desse cruzamento: **354 itens divergentes**.

6.5 A junção interna com condição composta e desigualdade

O **ON** aceita qualquer expressão booleana, não apenas igualdades:

```
-- Autos lavrados em ordens de serviço ainda não concluídas na data da lavratura
SELECT o.numero_os, a.numero_auto, o.data_abertura,
       a.data_lavratura, o.data_conclusao
FROM   dbo.ordem_servico AS o
       INNER JOIN dbo.auto_infracao AS a
         ON a.id_os = o.id_os
         AND a.data_lavratura >= o.data_abertura
         AND (o.data_conclusao IS NULL OR a.data_lavratura <= o.data_conclusao);
```

7. LEFT OUTER JOIN — preservando o lado esquerdo

7.1 Semântica

O **LEFT OUTER JOIN** retorna **todas as linhas da tabela à esquerda**, tenham par ou não. Onde não há par, as colunas da tabela da direita vêm preenchidas com **NULL**.

```
Esquerda: A B C D      LEFT JOIN em X:
Direita:   B C E      resultado = A(NULL) B C D(NULL)
```

Na fiscalização, esta é a **junção da omissão**. Toda pergunta que começa com "quem *não*..." é uma junção externa à esquerda:

- Quem **não** escriturou a nota que emitiu?
- Qual contribuinte **nunca** foi alvo de ordem de serviço?
- Qual ordem de serviço concluída **não** gerou auto de infração?
- Qual auditor **não** tem processo em andamento?

7.2 Exemplo: ordens de serviço com e sem auto de infração

```

SELECT  o.numero_os,
        o.tipo_fiscalizacao,
        o.situacao,
        a.numero_auto,                -- NULL quando não houve auto
        a.valor_principal,
        a.valor_multa
FROM    dbo.ordem_servico AS o
        LEFT OUTER JOIN dbo.auto_infracao AS a
            ON a.id_os = o.id_os
ORDER BY o.numero_os;

```

As 150 ordens de serviço aparecem todas. As que produziram auto trazem os dados do auto; as demais trazem **NULL** nas colunas de **a**. Trocando **LEFT** por **INNER**, o resultado encolhe para apenas as ordens com auto — e a pergunta "quantas fiscalizações terminaram sem autuação?" fica sem resposta.

7.3 Exemplo: consumidor não identificado

```

SELECT  n.chave_acesso,
        n.data_emissao,
        n.valor_total,
        d.cnpj AS cnpj_destinatario,
        d.razao_social AS destinatario,
        CASE WHEN n.id_destinatario IS NULL
            THEN 'CONSUMIDOR NAO IDENTIFICADO'
            ELSE 'IDENTIFICADO' END AS tipo_destinatario
FROM    dbo.nfe AS n
        LEFT OUTER JOIN dbo.contribuinte AS d
            ON d.id_contribuinte = n.id_destinatario
WHERE   n.situacao = 'AUTORIZADA';

```

Aqui o **LEFT JOIN** é **obrigatório**, não opcional. A coluna **id_destinatario** é anulável — 1.041 notas da base são para consumidor não identificado. Um **INNER JOIN** descartaria essas 1.041 notas silenciosamente, e todo somatório de faturamento sairia subestimado. Este é o erro de auditoria mais comum e mais caro que uma junção mal escolhida produz.

7.4 A antijunção: o **LEFT JOIN ... IS NULL**

Se o **LEFT JOIN** mantém as linhas sem par com **NULL** do lado direito, então **filtrar por esse NULL isola exatamente as linhas sem par**:

```

-- NF-e de saída AUTORIZADA que o emitente não escriturou na EFD
-- Resultado esperado: 2.023 documentos
SELECT  n.chave_acesso, n.numero, n.data_emissao,
        n.id_emitente, n.valor_total, n.valor_icms
FROM    dbo.nfe AS n
        LEFT OUTER JOIN dbo.efd_c100 AS e
            ON e.chave_acesso = n.chave_acesso
            AND e.id_contribuinte = n.id_emitente
            AND e.ind_oper = '1'
            AND e.cod_situacao = '00'
WHERE   n.situacao = 'AUTORIZADA'
        AND n.tipo_operacao = '1'
        AND e.id_c100 IS NULL;                -- <<< a antijunção

```

Três pontos de método, todos importantes:

1. **Todas as condições da correspondência ficam no ON.** Se **e.ind_oper = '1'** fosse para o **WHERE**, a consulta deixaria de ser externa (ver seção 10).
2. **O IS NULL deve ser testado numa coluna NOT NULL da tabela da direita** — aqui, a chave primária **e.id_c100**. Testar **e.chave_acesso IS NULL** seria ambíguo, porque essa coluna é anulável por natureza: um **NULL** ali pode significar "não houve par" ou "houve par, e o par tem chave nula".

3. **Filtre o regime tributário antes de acusar.** Contribuintes do Simples Nacional e MEI não entregam EFD ICMS/IPI. Sem esse filtro, aparecem 2.023 casos; restringindo a declarantes do regime **NORMAL**, restam **980** — os outros 1.043 são falsos positivos. Como observa o script 07, essa diferença é o achado didático mais importante da base: *a consulta tecnicamente correta pode estar juridicamente errada.*

```
-- A mesma antijunção, agora sem falso positivo: 980 documentos
SELECT COUNT(*) AS omissoes_regime_normal
FROM   dbo.nfe AS n
      INNER JOIN dbo.contribuinte AS c
            ON c.id_contribuinte = n.id_emitente
      LEFT OUTER JOIN dbo.efd_c100 AS e
            ON e.chave_acesso = n.chave_acesso
            AND e.id_contribuinte = n.id_emitente
            AND e.ind_oper = '1'
            AND e.cod_situacao = '00'
WHERE  n.situacao = 'AUTORIZADA'
      AND n.tipo_operacao = '1'
      AND c.regime_tributario = 'NORMAL'
      AND e.id_c100 IS NULL;
```

7.5 Encadeando junções externas

Uma vez que uma junção externa entra na consulta, **as junções seguintes que dependem do lado preservado também precisam ser externas** — do contrário, o **INNER JOIN** seguinte descarta as linhas cujo par veio nulo:

```
-- Contribuinte -> OS (pode não haver) -> auto (pode não haver)
SELECT c.cnpj, c.razao_social,
       o.numero_os, o.tipo_fiscalizacao,
       a.numero_auto, a.valor_multa
FROM   dbo.contribuinte AS c
      LEFT JOIN dbo.ordem_servico AS o ON o.id_contribuinte = c.id_contribuinte
      LEFT JOIN dbo.auto_infracao AS a ON a.id_os = o.id_os
ORDER BY c.razao_social, o.numero_os;
```

Se a segunda junção fosse **INNER JOIN**, todo contribuinte sem ordem de serviço sumiria — porque **o.id_os** viria **NULL** e nenhum auto casaria com **NULL**. O **INNER** posterior **anula** o **LEFT** anterior. Guarde essa frase: é uma das causas mais frequentes de "minha consulta perdeu linhas e eu não sei onde".

8. RIGHT OUTER JOIN — preservando o lado direito

8.1 Semântica

Espelho exato do **LEFT**: preserva todas as linhas da tabela **à direita**, completando com **NULL** o lado esquerdo quando não há par.

```
Esquerda: A B C D      RIGHT JOIN em X:
Direita:  B C E          resultado = B C E(NULL)
```

8.2 Toda **RIGHT** pode virar **LEFT**

As duas consultas abaixo são **logicamente idênticas**:

```
-- Com RIGHT
SELECT au.matricula, au.nome, o.numero_os
FROM   dbo.ordem_servico AS o
       RIGHT OUTER JOIN dbo.auditor_fiscal AS au
       ON au.matricula = o.matricula_auditor;

-- Com LEFT (mesma coisa, tabelas invertidas)
SELECT au.matricula, au.nome, o.numero_os
FROM   dbo.auditor_fiscal AS au
       LEFT OUTER JOIN  dbo.ordem_servico AS o
       ON o.matricula_auditor = au.matricula;
```

Ambas listam os 20 auditores, inclusive os que não têm ordem de serviço designada.

8.3 Então para que serve o **RIGHT JOIN**?

Na prática, para muito pouco — e essa é a conclusão honesta que o curso deve registrar. A recomendação profissional, que este material adota, é:

Padronize em **LEFT JOIN e leia a consulta sempre no mesmo sentido: da tabela principal para as complementares.**

O **RIGHT JOIN** tem, ainda assim, três situações em que aparece legitimamente:

1. **Ao acrescentar uma tabela ao final de uma consulta longa já pronta**, quando reordenar o **FROM** daria trabalho e risco. É uma conveniência de manutenção, não uma escolha de projeto.
2. **Ao ler código de terceiros** — inclusive SQL gerado por ferramentas de BI, que às vezes emitem **RIGHT JOIN**. É preciso saber traduzir mentalmente.
3. **Em consultas geradas dinamicamente**, em que a ordem das tabelas no **FROM** não está sob controle de quem escreve o predicado.

8.4 A armadilha de misturar **LEFT** e **RIGHT** na mesma consulta

```
-- Difícil de ler e fácil de errar. Evite.
SELECT ...
FROM   dbo.nfe AS n
LEFT JOIN  dbo.efd_c100 AS e      ON e.chave_acesso = n.chave_acesso
RIGHT JOIN dbo.contribuinte AS c  ON c.id_contribuinte = n.id_emitente;
```

Ao misturar os sentidos, "o lado preservado" deixa de ser óbvio: o **RIGHT JOIN** final preserva **contribuinte** contra **todo o resultado acumulado** das junções anteriores, não contra **nfe** isoladamente. Consultas assim passam em revisão porque ninguém consegue provar que estão erradas — e é exatamente esse o problema.

8.5 Um caso em que o **RIGHT JOIN** se lê bem

Quando a tabela de referência (a "lista de chamada") entra por último, o **RIGHT** preserva a naturalidade da leitura:

```
-- Quantas notas cada regime tributário emitiu, incluindo regimes sem nenhuma nota
SELECT c.regime_tributario, COUNT(n.id_nfe) AS qtd_notas
FROM   dbo.nfe AS n
       RIGHT JOIN  dbo.contribuinte AS c  ON c.id_contribuinte = n.id_emitente
GROUP BY c.regime_tributario;
```

Note o `COUNT(n.id_nfe)` e não `COUNT(*)`: em junção externa, `COUNT(*)` conta a linha fantasma preenchida com `NULL` e devolveria 1 onde o correto é 0. Este detalhe cai em prova — e cai em relatório de fiscalização.

9. FULL OUTER JOIN — a conciliação bidirecional

9.1 Semântica

Preserva **os dois lados**: linhas com par aparecem casadas; linhas sem par aparecem com `NULL` do lado que faltou.

```
Esquerda: A B C D      FULL JOIN em X:
Direita:  B C E         resultado = A(NULL) B C D(NULL) (NULL)E
```

9.2 O uso canônico: conciliar NF-e contra EFD nos dois sentidos

Este é o relatório de conciliação completo da malha fiscal. Ele responde, numa consulta só, às três perguntas que interessam:

```
SELECT COALESCE(n.chave_acesso, e.chave_acesso) AS chave_acesso,
CASE
  WHEN e.id_c100 IS NULL THEN 'EMITIDA E NAO ESCRITURADA'
  WHEN n.id_nfe IS NULL THEN 'ESCRITURADA SEM NF-e NA BASE'
  WHEN ABS(n.valor_total - e.valor_documento) > 0.01
       OR ABS(n.valor_icms - e.valor_icms) > 0.01
       THEN 'DIVERGENCIA DE VALOR'
  ELSE
    'CONCILIADA'
END AS resultado,
n.valor_total      AS valor_nfe,
e.valor_documento  AS valor_efd,
n.valor_icms       AS icms_nfe,
e.valor_icms       AS icms_efd
FROM   dbo.nfe AS n
FULL OUTER JOIN dbo.efd_c100 AS e
  ON e.chave_acesso = n.chave_acesso
WHERE  (n.id_nfe IS NULL OR n.situacao = 'AUTORIZADA')
AND    (e.id_c100 IS NULL OR e.chave_acesso IS NOT NULL);
```

O `COALESCE` na primeira coluna é indispensável: dependendo do lado que faltou, a chave só existe em um dos dois lados. Escrever apenas `n.chave_acesso` produziria `NULL` justamente nas linhas mais interessantes do relatório.

Os números de referência do script 07 permitem conferir o resultado:

Situação	Como se obtém	Esperado
Chaves de NF-e autorizada ausentes na EFD	<code>EXCEPT</code> (nfe – efd)	2.994
Chaves na EFD ausentes entre as NF-e autorizadas	<code>EXCEPT</code> (efd – nfe)	440
Chaves presentes nos dois lados	<code>INTERSECT</code>	6.897
Chaves na EFD que não existem em nenhuma NF-e da base	<code>NOT EXISTS</code>	38

Note a diferença entre as linhas 2 e 4: das 440 chaves escrituradas que não constam entre as NF-e **autorizadas**, apenas 38 não existem na base de forma alguma (documento de outra UF ou inexistente). As outras 402 existem — são notas **canceladas**, escrituradas corretamente com `cod_situacao = '02'`. É a

distinção entre "não achei" e "achei em outro estado", e ela separa uma intimação legítima de um constrangimento indevido ao contribuinte.

9.3 Contando cada bloco de uma vez

```
SELECT CASE WHEN e.id_c100 IS NULL THEN 'SO NA NF-e'
        WHEN n.id_nfe IS NULL THEN 'SO NA EFD'
        ELSE 'NOS DOIS' END AS lado,
COUNT(*) AS qtd
FROM dbo.nfe AS n
FULL OUTER JOIN dbo.efd_c100 AS e
    ON e.chave_acesso = n.chave_acesso
    AND n.situacao = 'AUTORIZADA'
GROUP BY CASE WHEN e.id_c100 IS NULL THEN 'SO NA NF-e'
            WHEN n.id_nfe IS NULL THEN 'SO NA EFD'
            ELSE 'NOS DOIS' END;
```

9.4 FULL OUTER JOIN × operadores de conjunto

EXCEPT, INTERSECT e UNION também comparam conjuntos, mas em outro nível:

	Junção externa	Operador de conjunto
Compara	linhas por um predicado arbitrário	listas de colunas posicionalmente compatíveis
Devolve	colunas dos dois lados	apenas as colunas da lista
Duplicatas	preservadas	eliminadas (exceto UNION ALL)
Permite ver os valores divergentes	Sim	Não, só a presença/ausência

Para "quais chaves faltam", EXCEPT é mais curto. Para "quais chaves faltam e quanto valem", só a junção externa serve.

10. ON ou WHERE? O erro que anula a junção externa

Este é o ponto do módulo que mais gera erro em produção. Leia com atenção.

Em uma junção interna, mover um predicado do ON para o WHERE não muda o resultado. Em uma junção externa, muda tudo.

10.1 O predicado no ON: filtra o que pode casar

```
-- CORRETO: lista TODAS as 150 ordens de serviço; para as concluídas em 2026,
-- traz o auto; para as demais, traz NULL.
SELECT o.numero_os, o.situacao, a.numero_auto
FROM   dbo.ordem_servico AS o
LEFT JOIN dbo.auto_infracao AS a
    ON a.id_os = o.id_os
    AND a.data_lavatura >= '2026-01-01';
```

O predicado no ON participa da decisão de quem casa com quem. Ordens sem auto em 2026 continuam no resultado, apenas sem par.

10.2 O mesmo predicado no **WHERE**: descarta a linha inteira

```
-- ERRADO (se a intenção era a de cima): o LEFT JOIN virou INNER JOIN na prática.
SELECT o.numero_os, o.situacao, a.numero_auto
FROM   dbo.ordem_servico AS o
       LEFT JOIN dbo.auto_infracao AS a
           ON a.id_os = o.id_os
WHERE  a.data_lavratuara >= '2026-01-01';
```

O **WHERE** é aplicado **depois** da junção. As linhas sem par têm **a.data_lavratuara = NULL**, e **NULL >= '2026-01-01'** não é verdadeiro — é *desconhecido*. Logo, todas as linhas sem par são descartadas e a junção externa se degrada em interna.

Regra prática:

Em junção externa, tudo o que se refere à **tabela preservada** vai no **WHERE**; tudo o que se refere à **tabela opcional** vai no **ON**. A única exceção é o teste **IS NULL** da antijunção, que é feito de propósito no **WHERE**.

10.3 Demonstração numérica

Execute as três e compare as contagens:

```
-- (a) Todas as OS, com auto quando houver
SELECT COUNT(*) FROM dbo.ordem_servico AS o
       LEFT JOIN dbo.auto_infracao AS a ON a.id_os = o.id_os;

-- (b) Todas as OS, casando só com autos acima de R$ 50.000
SELECT COUNT(*) FROM dbo.ordem_servico AS o
       LEFT JOIN dbo.auto_infracao AS a
           ON a.id_os = o.id_os AND a.valor_principal > 50000;

-- (c) Só as OS que têm auto acima de R$ 50.000 (LEFT anulado pelo WHERE)
SELECT COUNT(*) FROM dbo.ordem_servico AS o
       LEFT JOIN dbo.auto_infracao AS a ON a.id_os = o.id_os
WHERE  a.valor_principal > 50000;
```

As contagens de (a) e (b) devem ser **iguais** — o predicado adicional no **ON** não elimina linhas, apenas troca pares por **NULL**. A de (c) é bem menor. Se você entendeu por que, entendeu junção externa.

11. Antijunção e semijunção

11.1 As quatro formas de perguntar "existe par?"

Objetivo	Forma	Observação
Existe par (semijunção)	WHERE EXISTS (SELECT 1 ...)	Não duplica linhas
Existe par	INNER JOIN + DISTINCT	Duplica antes de deduplicar; evite
Não existe par (antijunção)	WHERE NOT EXISTS (SELECT 1 ...)	Forma preferida
Não existe par	LEFT JOIN ... WHERE chave IS NULL	Equivalente; útil para ver colunas do lado direito
Não existe par	WHERE col NOT IN (SELECT ...)	 Perigoso com NULL

11.2 O perigo do NOT IN

```
-- ARMADILHA: se a subconsulta retornar ao menos um NULL, o resultado é VAZIO.
SELECT COUNT(*)
FROM   dbo.contribuinte AS c
WHERE  c.id_contribuinte NOT IN (SELECT n.id_destinatario FROM dbo.nfe AS n);
```

Como `nfe.id_destinatario` é anulável e há 1.041 notas para consumidor não identificado, a subconsulta devolve `NULL`s. A comparação `id <> NULL` é *desconhecida* para toda linha, e a consulta retorna **zero** — silenciosamente, sem erro, sem aviso. A versão correta:

```
-- Esperado: 7 contribuintes que nunca figuraram como destinatários
SELECT COUNT(*) AS nunca_destinatarios
FROM   dbo.contribuinte AS c
WHERE  NOT EXISTS (SELECT 1 FROM dbo.nfe AS n
                  WHERE n.id_destinatario = c.id_contribuinte);
```

`NOT EXISTS` trata `NULL` corretamente porque não compara valores: apenas verifica se a subconsulta produziu alguma linha.

11.3 Semijunção: filtrar sem trazer colunas

Quando você precisa **filtrar** por existência mas não quer as colunas da outra tabela, `EXISTS` é mais preciso que `JOIN`:

```
-- Contribuintes ATIVOS que emitiram ao menos uma nota acima de R$ 100.000
-- Cada contribuinte aparece UMA vez, mesmo tendo 50 notas assim.
SELECT c.cnpj, c.razao_social
FROM   dbo.contribuinte AS c
WHERE  c.situacao_cadastral = 'ATIVO'
      AND EXISTS (SELECT 1 FROM dbo.nfe AS n
                  WHERE n.id_emitente = c.id_contribuinte
                        AND n.situacao = 'AUTORIZADA'
                        AND n.valor_total > 100000);
```

Com `INNER JOIN`, o mesmo contribuinte apareceria uma vez por nota qualificada, exigindo `DISTINCT` — que custa uma ordenação ou hash desnecessários.

11.4 Omissos de obrigação acessória: CROSS JOIN + antijunção

Aqui as duas técnicas se encontram. A grade da seção 5 fornece a lista do que *deveria* existir; a antijunção aponta o que falta:

```
-- Quem, entre os ATIVOS do regime NORMAL, deixou de entregar cada período de 2025
WITH grade AS (
  SELECT c.id_contribuinte, c.cnpj, c.razao_social, p.periodo_apuracao
  FROM   dbo.contribuinte AS c
        CROSS JOIN (VALUES ('202501'),('202502'),('202503'),('202504'),
                           ('202505'),('202506'),('202507'),('202508'),
                           ('202509'),('202510'),('202511'),('202512'))
        ) AS p(periodo_apuracao)
  WHERE  c.situacao_cadastral = 'ATIVO'
        AND c.regime_tributario = 'NORMAL'
)
SELECT g.cnpj, g.razao_social, g.periodo_apuracao
FROM   grade AS g
      LEFT JOIN dbo.efd_c100 AS e
        ON e.id_contribuinte = g.id_contribuinte
       AND e.periodo_apuracao = g.periodo_apuracao
```

```
WHERE e.id_c100 IS NULL
ORDER BY g.razao_social, g.periodo_apuracao;
```

Para conferência parcial: o script 07 registra **8 contribuintes ATIVOS** do regime **NORMAL** sem nenhuma EFD do período **202501**.

12. SELF JOIN — a tabela consigo mesma

Não é um tipo novo de junção — é o uso de **INNER**, **LEFT** ou **FULL** com a **mesma tabela nos dois lados**, distinguida por aliases diferentes. Sem alias, o comando é sintaticamente ambíguo e o SQL Server recusa.

12.1 Hierarquia de auditores

```
SELECT a.matricula,
       a.nome      AS auditor,
       a.cargo,
       a.regiao_fiscal,
       s.matricula AS matricula_supervisor,
       s.nome      AS supervisor,
       s.cargo      AS cargo_supervisor
FROM   dbo.auditor_fiscal AS a
       LEFT JOIN dbo.auditor_fiscal AS s
           ON s.matricula = a.matricula_supervisor
ORDER BY s.nome, a.nome;
```

LEFT, e não **INNER**: a gerente ANA BEATRIZ COSTA (matrícula 1001) tem **matricula_supervisor** nulo por estar no topo da hierarquia. Um **INNER JOIN** a excluiria do organograma — e o organograma sem o topo é um organograma errado.

Para três níveis, encadeiam-se dois autorrelacionamentos:

```
SELECT a.nome AS auditor, s.nome AS supervisor, g.nome AS gerente
FROM   dbo.auditor_fiscal AS a
       LEFT JOIN dbo.auditor_fiscal AS s ON s.matricula = a.matricula_supervisor
       LEFT JOIN dbo.auditor_fiscal AS g ON g.matricula = s.matricula_supervisor
WHERE  a.cargo = 'AUDITOR'
ORDER BY g.nome, s.nome, a.nome;
```

Para hierarquias de profundidade indefinida, a ferramenta correta é a CTE recursiva (**WITH ... UNION ALL**), assunto de módulo próprio. Autojunções encadeadas só funcionam quando o número de níveis é conhecido de antemão.

12.2 Operações circulares entre contribuintes

Esta é a autojunção mais interessante da base — e um indício clássico de simulação de operações. A pergunta é: existem pares de contribuintes que emitem notas **um para o outro**, em curto intervalo?

```
SELECT n1.id_emitente AS contribuinte_a,
       n1.id_destinatario AS contribuinte_b,
       COUNT(DISTINCT n1.id_nfe) AS notas_a_para_b,
       COUNT(DISTINCT n2.id_nfe) AS notas_b_para_a
FROM   dbo.nfe AS n1
       INNER JOIN dbo.nfe AS n2
           ON n2.id_emitente = n1.id_destinatario
           AND n2.id_destinatario = n1.id_emitente
           AND ABS(DATEDIFF(DAY, n1.data_emissao, n2.data_emissao)) <= 30
WHERE  n1.situacao = 'AUTORIZADA'
       AND n2.situacao = 'AUTORIZADA'
       AND n1.id_emitente < n1.id_destinatario -- evita listar o par duas vezes
GROUP BY n1.id_emitente, n1.id_destinatario
```

```
HAVING COUNT(DISTINCT n1.id_nfe) >= 3
AND COUNT(DISTINCT n2.id_nfe) >= 3
ORDER BY notas_a_para_b DESC;
```

Três técnicas se combinam aqui:

- **Autojunção invertida:** o **ON** cruza emitente com destinatário *trocados*, que é a definição algébrica de "circularidade".
- **Predicado de desigualdade** `n1.id_emitente < n1.id_destinatario`: sem ele, o par (8, 19) apareceria também como (19, 8). É o padrão para deduplicar pares simétricos.
- **COUNT(DISTINCT ...)** **obrigatório**: a junção multiplica as combinações de notas (fan-out, seção 15). `COUNT(*)` contaria pares de notas, não notas.

Resultado esperado pelo script 07: **3 pares** — contribuintes (8, 19), (14, 27) e (23, 31).

12.3 Sócios em comum (script 09)

Se a tabela `socio_administrador` estiver carregada, a autojunção por CPF revela vínculo societário entre empresas distintas — o chamado "grupo econômico de fato":

```
SELECT s1.cpf, s1.nome,
       s1.id_contribuinte AS empresa_1,
       s2.id_contribuinte AS empresa_2
FROM   dbo.socio_administrador AS s1
       INNER JOIN dbo.socio_administrador AS s2
           ON s2.cpf = s1.cpf
          AND s2.id_contribuinte > s1.id_contribuinte;
```

⚠ Esta consulta **vai devolver menos vínculos do que existem**. A coluna `cpf` foi propositalmente gravada com formatos heterogêneos (com máscara, sem máscara, com espaços, com zero à esquerda perdido). Junção por texto sujo é junção que perde par. A normalização do dado antes do cruzamento é tema do módulo de tratamento de dados; aqui ela serve como advertência: **antes de confiar numa junção, verifique se as duas colunas falam a mesma língua**.

13. Junção theta (não equijunção)

Quando o predicado de junção usa `<`, `>`, `BETWEEN` ou qualquer comparação que não seja igualdade, a operação se chama **junção theta**. A equijunção é apenas o caso particular em que theta é `=`.

13.1 Preço praticado contra a pauta fiscal

A tabela `referencia_preco` guarda a faixa aceitável por NCM. O cruzamento é por igualdade no NCM e por **faixa** no valor:

```
-- Itens vendidos abaixo do piso da pauta (indício de subfaturamento)
SELECT n.chave_acesso,
       c.cnpj,
       c.razao_social,
       i.descricao,
       i.ncm,
       r.descricao AS produto_pauta,
       i.valor_unitario,
       r.valor_min,
       r.valor_max,
       CAST((r.valor_min - i.valor_unitario) / NULLIF(r.valor_min,0) * 100
           AS DECIMAL(6,2)) AS pct_abaixo_da_pauta
FROM   dbo.nfe_item AS i
       INNER JOIN dbo.nfe AS n ON n.id_nfe = i.id_nfe
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
```

```

INNER JOIN dbo.referencia_preco AS r
      ON r.ncm = i.ncm -- equijunção
      AND i.valor_unitario < r.valor_min -- theta
WHERE n.situacao = 'AUTORIZADA'
      AND n.tipo_operacao = '1'
ORDER BY pct_abaixo_da_pauta DESC;

```

Invertendo o sinal, obtém-se o indício oposto — venda acima do teto, que pode indicar erro de digitação, produto diferente do declarado ou NCM incorreto:

```

AND i.valor_unitario > r.valor_max

```

E para separar os três casos numa passagem só, a junção volta a ser simples e a classificação vai para o **SELECT**:

```

SELECT i.ncm, COUNT(*) AS qtd_itens,
      SUM(CASE WHEN i.valor_unitario < r.valor_min THEN 1 ELSE 0 END) AS abaixo,
      SUM(CASE WHEN i.valor_unitario BETWEEN r.valor_min AND r.valor_max
            THEN 1 ELSE 0 END) AS dentro,
      SUM(CASE WHEN i.valor_unitario > r.valor_max THEN 1 ELSE 0 END) AS acima
FROM   dbo.nfe_item AS i
      INNER JOIN dbo.referencia_preco AS r ON r.ncm = i.ncm
GROUP BY i.ncm
ORDER BY abaixo DESC;

```

13.2 Junção por intervalo de datas

O mesmo princípio aplicado ao tempo: relacionar cada auto de infração com a ordem de serviço vigente na data da lavratura.

```

SELECT a.numero_auto, a.data_lavratura,
      o.numero_os, o.data_abertura, o.data_conclusao
FROM   dbo.auto_infracao AS a
      INNER JOIN dbo.ordem_servico AS o
            ON o.id_contribuinte = a.id_contribuinte
            AND a.data_lavratura BETWEEN o.data_abertura
            AND COALESCE(o.data_conclusao, '9999-12-31');

```

O **COALESCE** resolve o caso da ordem ainda em andamento (63 delas, sem **data_conclusao**). Sem ele, o **BETWEEN** contra **NULL** seria desconhecido e essas ordens desapareceriam.

⚠ **Junções theta não usam índice com a mesma eficiência que equijunções.** O SQL Server não pode aplicar *hash join* a um predicado de desigualdade — sobra-lhe *nested loops* ou *merge*. Em tabelas grandes, combine sempre a theta com ao menos uma igualdade seletiva (aqui, **r.ncm = i.ncm**) que restrinja o universo antes da comparação por faixa.

14. CROSS APPLY e OUTER APPLY

APPLY é uma extensão do T-SQL (não é padrão ANSI) que junta cada linha da esquerda a uma **subconsulta correlacionada** — algo que o **JOIN** comum não permite, porque no **JOIN** o lado direito não pode referenciar colunas do lado esquerdo.

Operador	Correspondência	Linhas sem par
CROSS APPLY	como INNER JOIN	descartadas
OUTER APPLY	como LEFT JOIN	preservadas com NULL

14.1 Top-N por grupo: as 3 maiores notas de cada contribuinte

```
SELECT c.cnpj, c.razao_social,
       t.chave_acesso, t.data_emissao, t.valor_total
FROM   dbo.contribuinte AS c
CROSS APPLY (
    SELECT TOP (3) n.chave_acesso, n.data_emissao, n.valor_total
    FROM   dbo.nfe AS n
    WHERE  n.id_emitente = c.id_contribuinte -- correlação: só APPLY permite
           AND n.situacao = 'AUTORIZADA'
    ORDER BY n.valor_total DESC
) AS t
ORDER BY c.razao_social, t.valor_total DESC;
```

Trocando para **OUTER APPLY**, os contribuintes sem nenhuma nota autorizada continuam listados, com **NULL** nas colunas do documento.

14.2 Última ocorrência: a fiscalização mais recente de cada contribuinte

```
SELECT c.cnpj, c.razao_social,
       ult.numero_os, ult.data_abertura, ult.situacao
FROM   dbo.contribuinte AS c
OUTER APPLY (
    SELECT TOP (1) o.numero_os, o.data_abertura, o.situacao
    FROM   dbo.ordem_servico AS o
    WHERE  o.id_contribuinte = c.id_contribuinte
    ORDER BY o.data_abertura DESC
) AS ult
ORDER BY ult.data_abertura DESC;
```

Sem **APPLY**, isso exigiria uma função de janela (**ROW_NUMBER()**) numa subconsulta, ou uma junção contra uma agregação por **MAX(data_abertura)** — que dá empate quando há duas ordens no mesmo dia. O **OUTER APPLY** com **TOP (1)** é mais direto e mais legível.

15. Junção e agregação: o problema do fan-out

15.1 O sintoma

```
-- CONSULTA ERRADA: o valor total sai inflacionado
SELECT c.razao_social,
       SUM(n.valor_total) AS total_notas,      -- <<< ERRADO
       COUNT(DISTINCT n.id_nfe) AS qtd_notas
FROM   dbo.contribuinte AS c
INNER JOIN dbo.nfe AS n ON n.id_emitente = c.id_contribuinte
INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
WHERE  n.situacao = 'AUTORIZADA'
GROUP BY c.razao_social;
```

O erro é sutil e não gera mensagem alguma. Ao juntar **nfe** com **nfe_item**, **cada cabeçalho de nota é repetido uma vez por item**. Uma nota de R\$1.000 com 4 itens vira quatro linhas de R\$1.000, e **SUM(n.valor_total)** devolve R\$ 4.000. Com 10.636 notas e 30.794 itens, o total sai aproximadamente **três vezes maior** que o correto.

Esse é o *fan-out*: a junção 1:N multiplica as linhas do lado "1".

15.2 As três correções

(a) **Agregar antes de juntar** — a forma mais clara e a que se recomenda:

```
WITH tot_nfe AS (  
    SELECT n.id_emitente,  
           SUM(n.valor_total) AS total_notas,  
           COUNT(*)           AS qtd_notas  
    FROM   dbo.nfe AS n  
    WHERE  n.situacao = 'AUTORIZADA'  
    GROUP BY n.id_emitente  
)  
tot_item AS (  
    SELECT n.id_emitente,  
           SUM(i.valor_total_item) AS total_itens,  
           COUNT(*)               AS qtd_itens  
    FROM   dbo.nfe AS n  
           INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe  
    WHERE  n.situacao = 'AUTORIZADA'  
    GROUP BY n.id_emitente  
)  
SELECT c.cnpj, c.razao_social,  
       t.qtd_notas, t.total_notas,  
       it.qtd_itens, it.total_itens,  
       t.total_notas - it.total_itens AS diferenca  
FROM   dbo.contribuinte AS c  
       LEFT JOIN tot_nfe AS t ON t.id_emitente = c.id_contribuinte  
       LEFT JOIN tot_item AS it ON it.id_emitente = c.id_contribuinte  
ORDER BY ABS(COALESCE(t.total_notas,0) - COALESCE(it.total_itens,0)) DESC;
```

De quebra, essa consulta é um cruzamento fiscal de verdade: a coluna **diferenca** denuncia as notas em que a soma dos itens não bate com o cabeçalho. O script 07 registra **15 notas** nessa condição.

(b) **SUM(DISTINCT ...)** — só funciona quando os valores repetidos são de fato idênticos, e falha se duas notas do mesmo contribuinte tiverem exatamente o mesmo valor. **Não use como solução geral.**

(c) **Agregar sobre a chave** — $SUM(n.valor_total) / COUNT(*) * COUNT(DISTINCT n.id_nfe)$ e variantes: matematicamente possível, ilegível na prática. Evite.

15.3 Como detectar o fan-out antes de publicar o relatório

Antes de confiar em qualquer somatório sobre junção, execute a checagem de granularidade:

```
SELECT COUNT(*) AS linhas_apos_juncao, COUNT(DISTINCT n.id_nfe) AS notas_distintas  
FROM   dbo.nfe AS n  
       INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe;
```

Se **linhas_apos_juncao > notas_distintas**, qualquer agregação sobre colunas de **nfe** está **inflacionada**. A regra é: identifique a granularidade do resultado (uma linha por quê?) e verifique se ela é a mesma da coluna que você está somando.

16. NULL nas colunas de junção

Três comportamentos precisam estar claros:

1. **NULL nunca casa com NULL numa junção.**

```
-- Zero linhas: as 1.041 notas sem destinatário não casam entre si nem com nada  
SELECT COUNT(*)  
FROM   dbo.nfe AS n1
```

```
INNER JOIN dbo.nfe AS n2 ON n2.id_destinatario = n1.id_destinatario
WHERE n1.id_destinatario IS NULL;
```

Para casar nulos deliberadamente, use **INTERSECT**, **EXISTS** com **IS NULL**, ou o predicado explícito:

```
ON (a.col = b.col) OR (a.col IS NULL AND b.col IS NULL)
```

...ciente de que essa forma normalmente impede o uso de índice.

2. Junção externa produz NULL que não existe nos dados. Um **NULL** no resultado de um **LEFT JOIN** pode significar duas coisas diferentes: "não houve par" ou "houve par e o valor lá é nulo". Distinguir exige testar uma coluna **NOT NULL** do lado direito — normalmente a chave primária.

3. Agregações ignoram NULL, exceto COUNT(*).

```
SELECT c.razao_social,
       COUNT(*)        AS conta_tudo,      -- conta a linha fantasma: mínimo 1
       COUNT(o.id_os)   AS conta_os,       -- correto: 0 quando não há OS
       COUNT(a.id_auto) AS conta_autos
FROM   dbo.contribuinte AS c
       LEFT JOIN dbo.ordem_servico AS o ON o.id_contribuinte = c.id_contribuinte
       LEFT JOIN dbo.auto_infracao AS a ON a.id_os           = o.id_os
GROUP BY c.razao_social
ORDER BY conta_autos DESC;
```

Em relatório de fiscalização, a diferença entre **COUNT(*)** e **COUNT(coluna)** é a diferença entre dizer que um contribuinte tem uma fiscalização e dizer que ele não tem nenhuma.

17. Junções e desempenho

O SQL Server implementa a junção lógica com três algoritmos físicos. O otimizador escolhe; o analista precisa saber ler a escolha no plano de execução (**Ctrl+M** no SSMS, "Include Actual Execution Plan").

Algoritmo	Como funciona	Bom quando	Sinal de alerta
Nested Loops	Para cada linha da entrada externa, busca correspondências na interna	Entrada externa pequena e índice na interna	Entrada externa grande sem índice → custo explode
Merge Join	Percorre as duas entradas já ordenadas, em paralelo	Ambos os lados ordenados pela chave de junção	Um Sort caro aparece antes do operador
Hash Match	Constrói tabela hash do lado menor e sonda com o maior	Tabelas grandes, sem índice adequado	<i>Hash spill to tempdb</i> → memória insuficiente

Boas práticas que valem para esta base:

- Índice nas colunas de junção.** O script `01_criar_banco.sql` já cria os índices mínimos sobre as chaves estrangeiras mais usadas (**IX_nfe_emitente**, **IX_nfe_item_nfe**, **IX_efd_c170_c100** etc.). O script `08_indices_opcionais.sql` acrescenta os índices de cobertura — inclusive **IX_efd_c100_chave**, que transforma o cruzamento NF-e × EFD por chave de acesso.
- Não aplique função sobre a coluna de junção.** Um predicado como **ON LEFT(e.chave_acesso, 44) = n.chave_acesso** impede o uso do índice (o predicado deixa de ser **SARGable**).
- Cuidado com tipos diferentes.** Juntar **CHAR(44)** com **VARCHAR(44)** funciona, mas juntar texto com número força conversão implícita e descarta o índice. É outra razão para normalizar o CPF de **socio_administrador** antes de cruzar.

4. **Filtre cedo.** Quanto menor o conjunto que chega à junção, melhor. Predicados seletivos sobre a tabela preservada devem estar no **WHERE**; sobre a opcional, no **ON**.
5. **Junção externa custa mais que interna.** Use **LEFT JOIN** quando a semântica exigir — não por hábito. Um **LEFT JOIN** cujo resultado é depois filtrado por **WHERE coluna_direita = 'X'** é apenas um **INNER JOIN** mais caro e mais confuso.

O exercício comparativo de plano de execução — rodar a consulta, ler a sugestão de índice ausente, criar o índice do script 08 e comparar — pertence ao módulo de desempenho. Por ora, basta observar que **a mesma junção lógica pode ter custos radicalmente diferentes**.

18. Erros comuns — quadro de diagnóstico

Sintoma observado	Causa provável	Correção
Resultado com muito mais linhas que o esperado	Critério de junção incompleto (faltou num_item , por exemplo) ou ausente	Verifique se o ON cobre a chave inteira do lado N
Somatório absurdamente alto	Fan-out em junção 1:N	Agregue antes de juntar (seção 15)
LEFT JOIN que "não trouxe os nulos"	Predicado da tabela opcional colocado no WHERE	Mova o predicado para o ON (seção 10)
Contribuintes sumiram do relatório	INNER JOIN depois de LEFT JOIN na mesma cadeia	Torne externas todas as junções dependentes
Consulta de omissos retorna zero	NOT IN com subconsulta contendo NULL	Troque por NOT EXISTS
COUNT devolve 1 onde deveria ser 0	COUNT(*) sobre junção externa	Use COUNT(coluna_da_tabela_direita)
Chave nula no relatório de conciliação	SELECT n.chave_acesso em FULL JOIN	Use COALESCE(n.chave, e.chave)
Vínculos societários não encontrados	Junção por texto com formatos heterogêneos	Normalize o dado antes de cruzar
Consulta correta, conclusão errada	Falta de filtro de regime tributário	Simples/MEI não entregam EFD ICMS/IPI
Ambiguous column name	Coluna com mesmo nome nas duas tabelas, sem qualificação	Qualifique com o alias

19. Quadro resumo dos tipos de junção

Tipo	Sintaxe	Preserva	Pergunta que responde	Uso típico nesta base
Interna	INNER JOIN	nenhum lado	"O que existe nos dois?"	NF-e × emitente; item × C170
Externa à esquerda	LEFT [OUTER] JOIN	esquerda	"Tudo da esquerda, com o par se houver"	OS com/sem auto; nota com/sem destinatário
Antijunção	LEFT JOIN + IS NULL	esquerda sem par	"O que existe só à esquerda?"	NF-e não escriturada (2.023 / 980)

Tipo	Sintaxe	Preserva	Pergunta que responde	Uso típico nesta base
Externa à direita	RIGHT [OUTER] JOIN	direita	Espelho do LEFT	Reescreva como LEFT
Externa completa	FULL [OUTER] JOIN	ambos	"O que existe em cada lado e nos dois?"	Conciliação NF-e × EFD
Produto cartesiano	CROSS JOIN	—	"Todas as combinações"	Grade contribuinte × período
Autojunção	qualquer, com aliases	conforme o tipo	"Relação da tabela com ela mesma"	Hierarquia; operações circulares
Theta	ON com < , > , BETWEEN	conforme o tipo	"Correspondência por faixa"	Preço × pauta de referência
CROSS APPLY	CROSS APPLY (subconsulta)	esquerda com par	"Top-N por grupo"	3 maiores notas por contribuinte
OUTER APPLY	OUTER APPLY (subconsulta)	esquerda	"Top-N por grupo, mantendo os sem"	Última OS de cada contribuinte
Semijunção	WHERE EXISTS	esquerda	"Existe ao menos um?"	Emitiu nota acima de R\$ 100 mil

20. Roteiro prático de laboratório

Duração estimada: 2 horas **Ambiente:** SSMS ou VS Code + extensão MSSQL, conectado à instância do curso **Base:** `curso_integridade_fiscal`, carregada com os scripts 01 a 05

Como este laboratório está organizado

Cada atividade parte de uma **regra de verificação da auditoria fiscal** — o tipo de regra que a malha fiscal aplica sobre a base de documentos eletrônicos. A regra vem primeiro; a junção vem depois, como consequência técnica dela. A pergunta a responder nunca é "escreva um **LEFT JOIN**", e sim "qual junção sustenta esta regra?".

Cada atividade traz:

Seção	Conteúdo
Regra	O enunciado da verificação, na linguagem da fiscalização
Fundamento	Por que a regra existe e o que a violação indica
Junção exigida	O tipo de junção que a regra impõe, e por quê
Rascunho	Consulta parcialmente escrita, com lacunas numeradas <code>/* (n) ... */</code>
Conferência	O número que a consulta correta deve produzir
Análise	Pergunta a responder em comentário, dentro do próprio script

Entrega. Um único arquivo `lab_juncoes_<seu_nome>.sql`, com as cinco atividades separadas por comentários, as lacunas preenchidas e as respostas de análise escritas como comentário logo abaixo de cada consulta.

Preparação (5 min). Antes de começar, confirme que a base está íntegra:

```
USE curso_integridade_fiscal;
GO
SELECT 'nfe' AS tabela, COUNT(*) AS obtido, 10636 AS esperado FROM dbo.nfe
UNION ALL SELECT 'nfe_item', COUNT(*), 30794 FROM dbo.nfe_item
UNION ALL SELECT 'efd_c100', COUNT(*), 8059 FROM dbo.efd_c100
UNION ALL SELECT 'efd_c170', COUNT(*), 23158 FROM dbo.efd_c170;
```

Se alguma linha divergir, recarregue os scripts 01 a 05: todas as conferências deste laboratório dependem desses volumes.

Atividade 1 — Omissão de escrituração

Regra. Toda NF-e de saída em situação **AUTORIZADA**, cujo emitente esteja no regime tributário **NORMAL**, deve corresponder a um registro C100 regular (*cod_situacao = '00', ind_oper = '1'*) escriturado pelo próprio emitente. A ausência desse registro caracteriza omissão de escrituração.

Fundamento. O contribuinte do regime normal é obrigado à EFD ICMS/IPI. Se o documento foi autorizado pela SEFAZ mas não aparece na escrituração do emitente, ou houve falha na entrega da obrigação acessória, ou houve omissão deliberada de receita. O filtro de regime é parte da regra, não um detalhe técnico: contribuintes do Simples Nacional e do MEI não entregam essa escrituração e apareceriam como falso positivo.

Junção exigida. Antijunção — **LEFT JOIN** seguido de teste **IS NULL**. A regra pergunta pelo que **não** existe do lado direito, e nenhuma junção interna consegue responder isso.

Rascunho.

```
/* ATIVIDADE 1 - Omissão de escrituração ----- */
SELECT  c.cnpj,
        c.razao_social,
        n.chave_acesso,
        n.numero,
        n.data_emissao,
        n.valor_total,
        n.valor_icms
FROM      dbo.nfe AS n
        INNER JOIN  dbo.contribuinte AS c
                ON c.id_contribuinte = n.id_emitente
        LEFT JOIN   dbo.efd_c100 AS e
                ON   e.chave_acesso      = n.chave_acesso
                AND e.id_contribuinte = /* (1) o declarante deve ser o próprio emitente */
                AND e.ind_oper          = /* (2) indicador de operação de saída */
                AND e.cod_situacao      = /* (3) código de documento regular */
WHERE     n.situacao          = 'AUTORIZADA'
        AND n.tipo_operacao   = '1'
        AND c.regime_tributario = /* (4) regime obrigado à EFD ICMS/IPI */
        AND /* (5) o teste que isola as linhas sem par - use a PK de efd_c100 */
ORDER BY n.valor_icms DESC;
```

Conferência.

Variação	Esperado
Consulta acima, com COUNT(*) no lugar da lista de colunas	980
A mesma consultasem a lacuna (4) — isto é, sem filtro de regime	2.023

Análise. (a) Por que as lacunas (1), (2) e (3) precisam ficar na cláusula **ON** e não no **WHERE**? Descreva o que aconteceria com o resultado se fossem movidas. (b) Os 1.043 documentos que somem quando o filtro de regime entra são falsos positivos. Qual seria a consequência prática de intimar esses contribuintes?

Atividade 2 — Conciliação NF-e × EFD nos dois sentidos

Regra. Para cada chave de acesso, o valor do documento e o valor do ICMS declarados na EFD devem coincidir com os da NF-e autorizada correspondente, com tolerância de R\$ 0,01. Além disso, toda chave escriturada deve existir na base de documentos autorizados, e toda NF-e autorizada deve estar escriturada.

Fundamento. A regra tem três violações possíveis, e a conciliação precisa enxergar as três: documento emitido e não escriturado, documento escriturado sem NF-e correspondente na base, e documento presente nos dois lados com valores divergentes. Note que a segunda violação nem sempre é irregularidade — pode ser documento de outra UF, ausente da base local, ou nota cancelada escriturada corretamente com `cod_situacao = '02'`.

Junção exigida. **FULL OUTER JOIN**. Nenhuma junção que preserve um só lado responde às três perguntas de uma vez.

Rascunho.

```
/* ATIVIDADE 2 - Conciliação bidirecional ----- */
SELECT /* (1) a chave de acesso, vinda do lado que existir - use COALESCE */ AS chave_acesso,
CASE
  WHEN /* (2) não houve par na EFD */ THEN 'EMITIDA E NAO ESCRITURADA'
  WHEN /* (3) não houve par na NF-e */ THEN 'ESCRITURADA SEM NF-e NA BASE'
  WHEN ABS(n.valor_total - e.valor_documento) > 0.01
       OR ABS(n.valor_icms - e.valor_icms) > 0.01
       THEN 'DIVERGENCIA DE VALOR'
  ELSE
    THEN 'CONCILIADA'
END AS resultado,
COUNT(*) AS qtd,
SUM(ABS(COALESCE(n.valor_total,0) - COALESCE(e.valor_documento,0))) AS soma_diferencas
FROM
  dbo.nfe AS n
  FULL OUTER JOIN dbo.efd_c100 AS e
    ON e.chave_acesso = n.chave_acesso
   AND /* (4) restrinja o lado da NF-e às autorizadas, SEM perder as linhas
        que só existem na EFD - pense em qual cláusula suporta isso */
GROUP BY CASE
  WHEN /* repita aqui a mesma expressão do SELECT */ THEN 'EMITIDA E NAO ESCRITURADA'
  ...
END;
```

Em seguida, escreva as três consultas de conferência com operadores de conjunto:

```
-- (5) chaves de NF-e autorizada ausentes na EFD
SELECT COUNT(*) FROM ( SELECT chave_acesso FROM dbo.nfe WHERE /* ... */
EXCEPT
SELECT chave_acesso FROM dbo.efd_c100 WHERE /* ... */ ) AS t;

-- (6) chaves escrituradas ausentes entre as NF-e autorizadas
-- (7) chaves que NÃO existem em nenhuma NF-e da base, em qualquer situação
-- (dica: NOT EXISTS sem filtro de situação)
```

Conferência.

Verificação	Esperado
Chaves de NF-e autorizada ausentes na EFD	2.994
Chaves escrituradas ausentes entre as autorizadas	440
Chaves presentes nos dois lados (INTERSECT)	6.897
Chaves que não existem em nenhuma NF-e da base	38
Documentos conciliados com divergência de valor ou ICMS	147

Análise. Das 440 chaves da segunda linha, apenas 38 não existem de forma alguma na base. As outras 402 existem. (a) O que são elas? (b) Por que a distinção entre esses dois grupos é decisiva antes de emitir qualquer intimação? (c) Por que o **COALESCE** da lacuna (1) é indispensável neste relatório e não seria em um **INNER JOIN**?

Atividade 3 — Preço praticado contra a pauta fiscal

Regra. O valor unitário praticado em cada item de NF-e de saída deve estar contido na faixa `[valor_min, valor_max]` definida na pauta de referência para o respectivo NCM. Valor abaixo do piso é indício de subfaturamento; valor acima do teto é indício de erro de classificação fiscal. O painel de acompanhamento deve exibir os 28 NCM da pauta, inclusive os que não tiveram movimento no período.

Fundamento. A pauta fiscal é o parâmetro de valor que a administração tributária usa para presumir a base de cálculo quando o preço declarado destoa do mercado. A última frase da regra é a parte que costuma ser esquecida: um NCM da pauta **sem nenhum movimento** também é informação — pode indicar que um produto sujeito a controle deixou de ser declarado.

Junção exigida. Junção theta (a faixa é comparada com `<`, `>` ou `BETWEEN`) combinada com uma junção externa que **preserve a pauta**. Como `referencia_preco` entra depois de `nfe_item` no `FROM`, este é um dos poucos casos em que o `RIGHT JOIN` se lê com naturalidade — mas a versão com `LEFT JOIN` e as tabelas invertidas é igualmente aceitável.

Rascunho.

```
/* ATIVIDADE 3 - Aderência à pauta fiscal ----- */
SELECT  r.ncm,
        r.descricao,
        r.valor_min,
        r.valor_max,
        COUNT(/* (1) conte os ITENS, não as linhas - qual coluna evita contar 1
                quando não houve movimento algum? */) AS qtd_itens,
        SUM(CASE WHEN i.valor_unitario < r.valor_min THEN 1 ELSE 0 END) AS abaixo_da_pauta,
        SUM(CASE WHEN /* (2) dentro da faixa */          THEN 1 ELSE 0 END) AS dentro_da_pauta,
        SUM(CASE WHEN /* (3) acima do teto */            THEN 1 ELSE 0 END) AS acima_da_pauta,
        MIN(i.valor_unitario) AS menor_praticado
FROM      dbo.nfe_item AS i
        INNER JOIN dbo.nfe AS n
                ON n.id_nfe = i.id_nfe
        AND /* (4) restrinja às saídas autorizadas SEM excluir os NCM sem movimento */
        RIGHT JOIN dbo.referencia_preco AS r
                ON r.ncm = i.ncm
GROUP BY  r.ncm, r.descricao, r.valor_min, r.valor_max
ORDER BY  abaixo_da_pauta DESC;
```

Depois, escreva a consulta de detalhe: os itens **abaixo do piso**, com CNPJ e razão social do emitente, chave de acesso, descrição do produto, valor praticado, `valor_min` e o percentual de desvio, ordenados pelo maior desvio.

```
-- Detalhe do subfaturamento
SELECT  c.cnpj, c.razao_social, n.chave_acesso, i.descricao, i.ncm,
        i.valor_unitario, r.valor_min,
        CAST(/* (5) percentual de desvio abaixo do piso */ AS DECIMAL(6,2)) AS pct_abaixo
FROM      dbo.nfe_item AS i
        INNER JOIN dbo.nfe          AS n ON /* (6) */
        INNER JOIN dbo.contribuinte AS c ON /* (7) */
        INNER JOIN dbo.referencia_preco AS r
                ON r.ncm = i.ncm
        AND /* (8) a condição theta que caracteriza o subfaturamento */
WHERE     n.situacao = 'AUTORIZADA' AND n.tipo_operacao = '1'
ORDER BY  pct_abaixo DESC;
```

Conferência.

Verificação	Esperado
Linhas do painel (uma por NCM da pauta)	28
NCM cujo <code>qtd_itens</code> é zero	devem existir, e não podem sumir do painel
Soma de <code>abaixo</code> + <code>dentro</code> + <code>acima</code> em cada linha	igual a <code>qtd_itens</code> da mesma linha

Análise. (a) Se a lacuna (4) fosse escrita no **WHERE** em vez do **ON**, quantas linhas o painel passaria a ter e por quê? (b) Por que a junção theta desta atividade traz uma igualdade (**r.ncm = i.ncm**) junto da comparação por faixa? O que aconteceria com o desempenho se a igualdade fosse removida?

Atividade 4 — Consistência interna do documento

Regra. A soma dos valores dos itens de uma NF-e deve ser igual ao valor total declarado no cabeçalho, com tolerância de R\$ 0,01. Documento em que essa igualdade não se verifica está internamente inconsistente e não pode ser usado como prova sem diligência prévia.

Fundamento. É uma regra de consistência interna: não depende de nenhuma declaração do contribuinte, apenas do próprio documento. Também é a regra que expõe o erro técnico mais comum em relatórios de auditoria — somar uma coluna do cabeçalho depois de juntar com a tabela de itens, o que **multiplica** o valor pelo número de itens.

Junção exigida. Agregação antes da junção. Primeiro consolide **nfe_item** por documento; só então junte com **nfe**. O caminho inverso produz *fan-out*.

Rascunho.

```
/* ATIVIDADE 4a - Demonstração do fan-out (consulta ERRADA, execute e anote) */
SELECT SUM(n.valor_total) AS total_inflacionado,
       COUNT(*)           AS linhas_apos_juncao,
       COUNT(DISTINCT n.id_nfe) AS notas_distintas
FROM   dbo.nfe AS n
       INNER JOIN dbo.nfe_item AS i ON i.id_nfe = n.id_nfe
WHERE  n.situacao = 'AUTORIZADA';

/* ATIVIDADE 4b - Consulta CORRETA */
WITH itens AS (
  SELECT i.id_nfe,
         SUM(/* (1) coluna de valor do item */) AS total_itens,
         COUNT(*) AS qtd_itens
  FROM   dbo.nfe_item AS i
  GROUP BY i.id_nfe
)
SELECT c.cnpj,
       c.razao_social,
       n.chave_acesso,
       n.data_emissao,
       n.valor_total AS valor_cabecalho,
       it.total_itens,
       it.qtd_itens,
       /* (2) a diferença entre cabeçalho e soma dos itens */ AS diferenca
FROM   dbo.nfe AS n
       INNER JOIN dbo.contribuinte AS c ON c.id_contribuinte = n.id_emitente
       /* (3) qual junção liga 'nfe' à CTE 'itens' sem descartar
          notas que porventura não tenham item algum? */ itens AS it
       ON it.id_nfe = n.id_nfe
WHERE  /* (4) o predicado que caracteriza a inconsistência, com tolerância de R$ 0,01 */
ORDER BY ABS(n.valor_total - it.total_itens) DESC;
```

Conferência.

Verificação	Esperado
Notas com soma de itens divergente do cabeçalho	15
linhas_apos_juncao vs. notas_distintas em 4a	a primeira é bem maior — anote a razão entre elas
total_inflacionado de 4a vs. soma correta de valor_total	anote os dois e a proporção

Análise. (a) Qual foi a razão entre o total inflacionado e o total correto? Relacione esse número com a razão entre **linhas_apos_juncao** e **notas_distintas**. (b) Por que **COUNT(DISTINCT n.id_nfe)** na

consulta 4a devolve o número certo enquanto `SUM(n.valor_total)` não? (c) Repita a checagem de granularidade para o par `efd_c100 × efd_c170` e registre os dois números.

Atividade 5 — Circularidade de operações entre contribuintes

Regra. *Dois contribuintes que emitem, um para o outro, três ou mais notas autorizadas em intervalo inferior a 30 dias configuram indício de operação circular, hipótese de simulação de circulação de mercadoria para geração de crédito de ICMS. Cada par deve ser reportado uma única vez.*

Fundamento. Na circulação real de mercadoria, o fluxo tem direção: fornecedor → cliente. O fluxo que volta na mesma intensidade e no mesmo intervalo curto raramente corresponde a mercadoria física; costuma corresponder a documento emitido para gerar crédito. É um dos indicadores clássicos da malha fiscal.

Junção exigida. Autojunção de `nfe` consigo mesma, com o predicado **invertido** (emissor de um lado casando com destinatário do outro) e um predicado de desigualdade para não reportar o mesmo par duas vezes.

Rascunho.

```
/* ATIVIDADE 5 - Operações circulares ----- */
SELECT ca.cnpj AS cnpj_a,
       ca.razao_social AS contribuinte_a,
       cb.cnpj AS cnpj_b,
       cb.razao_social AS contribuinte_b,
       COUNT(DISTINCT n1.id_nfe) AS notas_a_para_b,
       COUNT(DISTINCT n2.id_nfe) AS notas_b_para_a,
       SUM(/* (1) por que somar aqui daria um valor errado?
            deixe esta coluna de fora e explique na análise */) AS nao_usar
FROM   dbo.nfe AS n1
       INNER JOIN dbo.nfe AS n2
            ON n2.id_emitente = /* (2) */
            AND n2.id_destinatario = /* (3) */
            AND ABS(DATEDIFF(DAY, n1.data_emissao, n2.data_emissao)) <= 30
       INNER JOIN dbo.contribuinte AS ca ON ca.id_contribuinte = n1.id_emitente
       INNER JOIN dbo.contribuinte AS cb ON cb.id_contribuinte = n1.id_destinatario
WHERE  n1.situacao = 'AUTORIZADA'
       AND n2.situacao = 'AUTORIZADA'
       AND /* (4) o predicado que impede reportar o par (A,B) e também (B,A) */
GROUP BY ca.cnpj, ca.razao_social, cb.cnpj, cb.razao_social
HAVING /* (5) os dois limites da regra: três ou mais notas em cada sentido */
ORDER BY notas_a_para_b DESC;
```

Conferência.

Verificação	Esperado
Pares reportados	3
Identificação dos pares (<code>id_contribuinte</code>)	(8, 19), (14, 27) e (23, 31)

Análise. (a) O que acontece com o resultado se o predicado da lacuna (4) for removido? (b) Por que as contagens precisam ser `COUNT(DISTINCT ...)` e não `COUNT(*)`? Relacione sua resposta com o fan-out da Atividade 4. (c) Deixamos a lacuna (1) sem uso: explique por que somar `valor_total` diretamente nesta consulta produziria um valor sem significado, e descreva como obter o valor correto trafegado entre os dois contribuintes.

Ficha de conferência

Atividade	Verificação	Esperado	Obtido
1	Omissões de escrituração, regime NORMAL	980	
1	Idem, sem filtro de regime	2.023	
2	EXCEPT (NF-e – EFD)	2.994	
2	EXCEPT (EFD – NF-e)	440	
2	INTERSECT	6.897	
2	Chaves inexistentes na base de NF-e	38	
2	Divergências de valor / ICMS	147	
3	Linhas do painel de pauta	28	
4	Notas com soma de itens divergente	15	
4	Razão entre total inflacionado e total correto	anotar	
5	Pares em operação circular	3	

Critérios de avaliação

Critério	Peso
Correção dos resultados (ficha de conferência)	40%
Adequação da junção escolhida à regra de auditoria enunciada	25%
Posicionamento correto dos predicados (ON × WHERE)	15%
Qualidade das respostas de análise	20%

21. Referências

Documentação oficial da Microsoft

- *FROM plus JOIN, APPLY, PIVOT (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/from-transact-sql>
- *SELECT (Transact-SQL)* — <https://learn.microsoft.com/pt-br/sql/t-sql/queries/select-transact-sql>
- *Joins (SQL Server)* — <https://learn.microsoft.com/pt-br/sql/relational-databases/performance/joins>
- *Query Processing Architecture Guide* — <https://learn.microsoft.com/pt-br/sql/relational-databases/query-processing-architecture-guide>
- *Discontinued Database Engine Functionality in SQL Server* (fim da sintaxe ***=** e **=***) — <https://learn.microsoft.com/en-us/sql/database-engine/discontinued-database-engine-functionality-in-sql-server>
- *Display an Actual Execution Plan* — <https://learn.microsoft.com/pt-br/sql/relational-databases/performance/display-an-actual-execution-plan>

Bibliografia

- ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson. (Capítulos de álgebra relacional e SQL — a junção como composição de produto cartesiano e seleção.)
- SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de Banco de Dados*. 7. ed. Rio de Janeiro: LTC. (Capítulos de SQL intermediário — junções externas e valores nulos.)
- DATE, C. J. *SQL and Relational Theory*. 3. ed. O'Reilly. (Discussão sobre o tratamento de **NULL** em junções.)

Legislação e normas técnicas de referência do domínio fiscal

- Ajuste SINIEF 07/05 e alterações — Nota Fiscal Eletrônica (NF-e) e estrutura da chave de acesso.
- Convênio ICMS 143/06 e Ajuste SINIEF 02/09 — Escrituração Fiscal Digital (EFD), registros do Bloco C.
- Guia Prático da EFD ICMS/IPI — Secretaria Especial da Receita Federal / CONFAZ (registros C100 e C170).

Scripts do curso utilizados neste módulo

Script	Função
01_criar_banco.sql	Estrutura, restrições e índices mínimos
02_inserir_cadastros.sql	Municípios, contribuintes, auditores, pauta
03_inserir_nfe.sql	NF-e e itens
04_inserir_efd.sql	EFD C100 e C170
05_inserir_fiscalizacao.sql	Ordens de serviço e autos de infração
07_verificacao.sql	Conferência da carga e mapa das inconsistências
08_indices_opcionais.sql	Índices de desempenho (módulo de plano de execução)
09_quadro_societario.SQL	Quadro societário (opcional, autojunção por CPF da seção 12.3)

Módulo elaborado para o Curso de Banco de Dados Relacional aplicado à Fiscalização Tributária — SEFAZ-PB. Todos os exemplos foram escritos e conferidos contra a base [curso_integridade_fiscal](#) gerada pelos scripts 01 a 05.